

GharFind AI: System Approach & Design Trade-offs

Autonomous Property-Discovery Agent for Home Buyers

Author: Khushi U Poonja

Date: July 4, 2026

Target: Engineering Assignment

1. Executive Summary

Finding a home is a complex multi-criteria optimization problem. In the Indian market, this is further shaped by specific housing metrics (such as BHK - Bedroom Hall Kitchen spacing), unique pricing denominations (Lakhs and Crores), and challenging traffic/transit commute profiles.

This project implements an Autonomous Property-Discovery Web Application (GharFind AI) built using a full-stack Next.js (React + TypeScript) framework. The core system combines a conversational chat interface for natural preference extraction, an autonomous agentic loop to coordinate tool execution, a deterministic multi-criteria utility ranker to evaluate match compatibility, and a premium glassmorphic dashboard to visualize results.

To ensure immediate usability by evaluators, the app operates in two modes: Mock Agent Mode (zero setup, works instantly using keyword heuristics and simulated thought logs) and Live LLM Mode (plugs into Gemini or OpenAI APIs with genuine LLM function calling).

2. System Architecture & Flow

The application follows a clean unidirectional flow:

- **Frontend Dashboard:** A responsive dashboard containing a Sidebar (API key configuration, workplace location override, and preference weights), a Chat Interface (complete with an accordion-style Agent Thought Log mapping out tool calls), and a Ranked Shortlist Panel showing compatibility scores in Indian notation.
- **Agentic Executor:** API Route that coordinates conversation history, extracts housing criteria, runs the tool execution loop, compiles data, and invokes the deterministic ranker.
- **Commute Calculator:** Uses the Haversine formula to calculate geographic distance between properties and the buyer's workplace, then applies transport mode coefficients (driving, transit, walking, bicycling) and heavy traffic delays (e.g. driving average at 25 km/h with 10 min congestion buffer) to estimate door-to-door commute times in Indian cities.

3. The Ranking Engine: Multi-Criteria Utility Theory

Rather than relying on the LLM to qualitatively rank properties (which is prone to hallucinations and mathematical inconsistency), we implement a Deterministic Utility Function that scores properties on a 0-100% compatibility scale. The score is a weighted average of five category sub-scores:

$$\text{Match Score} = \frac{\text{Sum}(\text{Sub-Score}_i * \text{Weight}_i)}{\text{Sum}(\text{Weight}_i)}$$

Where Weight_i represents the user-defined priority weight (integer 1-5, defaults to 3).

- **Price Match (S_price):** If Price <= Budget, score is 100 (with a small bonus for being under budget). If Price > Budget, the score drops linearly, penalizing listings exceeding the budget.
- **Space Match (S_space):** Evaluates BHK (Bedroom-Hall-Kitchen) and bathroom counts. A penalty of -30% is applied per missing BHK and -20% per missing bathroom.
- **School Match (S_school):** Computes the average rating of nearby schools (Primary, Middle, Senior School, scale 1-10). If the average meets the buyer's target, score is 100. Otherwise, it drops by 20% per point under the target.
- **Safety Match (S_safety):** Evaluates the neighborhood safety score (1-100). If it meets the target, score is 100. Otherwise, it drops by 3% per safety point under the target.
- **Commute Match (S_commute):** Computes estimated commute time. If it is under the target (default 30 mins), score is 100. For every minute exceeding the target, the score drops by 3%.

4. Key Engineering Trade-Offs

- **Custom Agent Loop vs. Frameworks:** We implemented a lightweight, custom TypeScript agent loop instead of using bulky orchestration libraries. This is transparent, easy to debug, has zero bundle-size bloat, and provides granular control over the tool logs shown to the user.
- **Deterministic Ranker vs. LLM Ranker:** We separated the math from the language: the system calculates the scores deterministically, and the LLM translates the scores into conversational justifications. This guarantees mathematical consistency, avoiding LLM pricing errors.
- **Client-Side vs. Server-Side Execution:** The agent runs inside Next.js API Routes, while preference states and configuration reside in React state. Running the LLM on the server side keeps API keys secure and allows server-side logging, while client-side state allows immediate visual updates.

5. Future Enhancements & Production Roadmap

- **Real-time API integrations:** Connect to platforms like MagicBricks or Housing.com for live listings, local school directories for school ratings, and Google Maps API with live Indian traffic routing.

- **Interactive Map Panel:** Embed a Mapbox or Leaflet component to show candidate pins, neighborhood boundaries (like Indiranagar, BKC, Sector 62), and transit paths.
- **Vastu Shastra Optimization:** Adding a secondary utility sub-score for properties meeting Vastu compliance requirements, which is a highly valued feature for many Indian home buyers.